Faculty Evaluation System

Northwest Samar State University (NWSSU)

Technical guide: how the system works, how data flows, and how the code is arranged

1. Overview and technology
2. Folder structure and file arrangement
3. System architecture
4. Admin login flow
5. Student and supervisor login flow
6. Session restore and the password gate
7. Student evaluation flow (SET, Annex A)
8. Supervisor evaluation flow (SEF, Annex B)
9. Admin dashboard: boot, data layer and sync
10. Script load order (critical)
11. Admin pages and modules
12. Question sets (versioned instrument)
13. Scoring and ratings (CMO 19)
14. Term scoping and reports
15. Firestore data model
16. Security rules
17. Important notes and maintenance tips

1. Overview and technology

The Faculty Evaluation System is a web application. Students rate their teachers, supervisors (department chairs/deans) rate the faculty in their departments, and administrators manage the records and produce the official CHED **CMO No. 19 s. 2025** reports (Annex C, Annex D and the Faculty Evaluation Report).

It has **three kinds of users**:

User	Logs in at	What they do
Administrator	/index.html (email + password)	Manages school years, students, teachers, subjects, departments, courses, question sets and the evaluation period; views results; prints reports.
Student	/Student-user/index.html (Student ID + password)	Evaluates each teacher of each enrolled subject using the SET form (Annex A, 15 questions).
Supervisor	/Student-user/index.html (Teacher ID + password)	A teacher record with facultyType = "supervisor". Evaluates faculty in the departments they supervise using the SEF form (Annex B).

Technology stack

- **Plain HTML, CSS and JavaScript.** No framework, no build step, no npm. Each page loads its scripts with <script> tags.
- **Firebase JS SDK 10.12 (compat version)** loaded from gstatic.com: *firebase-app*, *firestore*, *auth* (and *functions* on the admin page).
- **Cloud Firestore** is the database. **Firebase Authentication** handles sign-in. **Firebase Hosting** serves the site (firebase.json sets the repo root "." as the public folder).
- **SheetJS (xlsx 0.18.5)** on the admin page for Excel export / import.
- Firebase project ID: **studenteval-937f6** (in .firebaserc). An Android app (separate code, "AuthRepository.kt") uses the same project and the same login conventions.

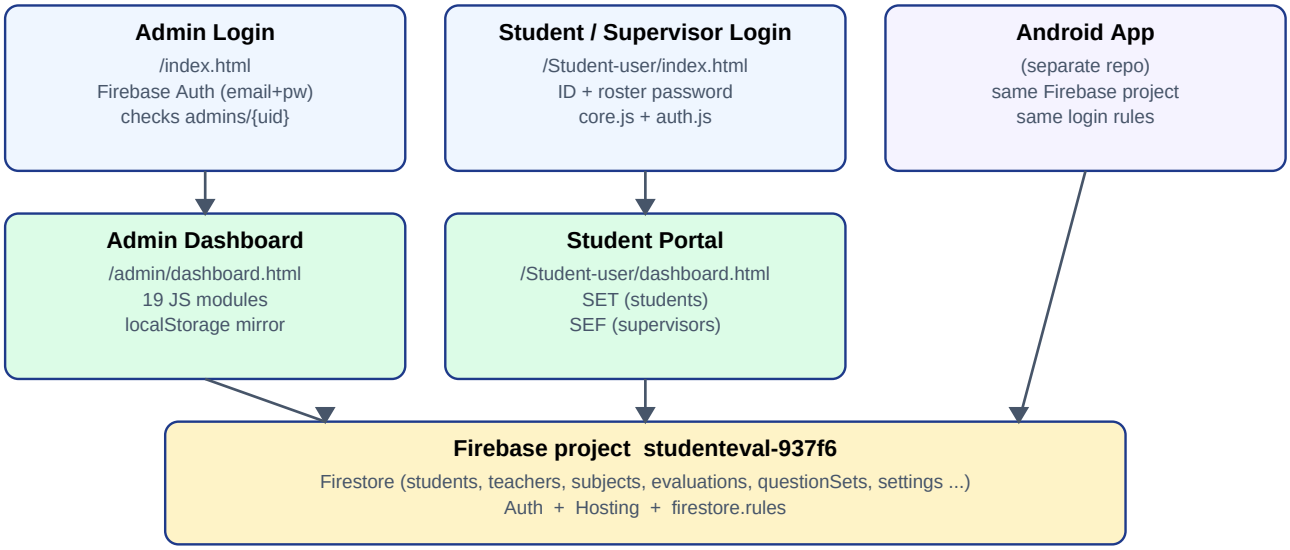
2. Folder structure and file arrangement

```
Faculty-Eval-System/
|-- index.html            Admin login page (entry point of the site)
|-- index.css             Styles for the admin login page
|-- 404.html              Not-found page for Firebase Hosting
|-- firebase.json         Hosting + Firestore rules config
|-- .firebaserc           Firebase project id (studenteval-937f6)
|-- firestore.rules       Database security rules
|-- shared/
|   |-- js/firebaseAPI.js Shared Firebase web config (window.firebaseConfig)
|   `-- css/styles.css    Shared styles
|-- icons/                Logos and department images (CCIS, CEA, COED ...)
|-- Student-user/         STUDENT + SUPERVISOR PORTAL
|   |-- index.html        Login page (ID + password, forced password gate)
|   |-- dashboard.html    The portal app (dashboard, evaluate, history ...)
|   |-- terms.html        Terms page
|   |-- css/user.css      Portal styles
|   `-- js/
|       |-- core.js       Firebase init, shared state, questions, helpers
|       |-- auth.js       Login, provisioning, password change, session restore
|       `-- app.js        Screens: dashboard, SET/SEF forms, history, profile
`-- admin/                ADMIN DASHBOARD
    |-- dashboard.html    Single-page admin app (all pages are <div>s)
    |-- admin-styles.css  Admin styles
    `-- js/
        |-- core/         admin-core, admin-scoring, admin-nav, admin-router
        |-- records/      students, teachers, subjects, bulk CSV, courses
        |-- reports/      reports, annex (C/D), fer, sef (+ audit log)
        `-- modules/      admindept, adminrate, adminEval,
                          adminfeatures, adminQuestions
```

How the arrangement works

- **Two separate apps share one Firebase project.** The admin app lives under */admin*; the student/supervisor app lives under */Student-user*. They never call each other directly; they meet only in Firestore.
- **Each app is split into a login page and a dashboard page.** The login page signs in and then redirects (location.replace) to the dashboard, so the Back button does not return to the login form.
- **The admin JS is grouped by job**: *core/* (storage, scoring, navigation, URL routing), *records/* (CRUD for people, subjects and courses), *reports/* (printed forms and analytics), *modules/* (departments, sync, term logic, features, questions). These files were split from one original 4,441-line admin.js.
- **The student JS is three layers**: core.js (data and helpers) -> auth.js (who you are) -> app.js (what you see). They must load in that order.

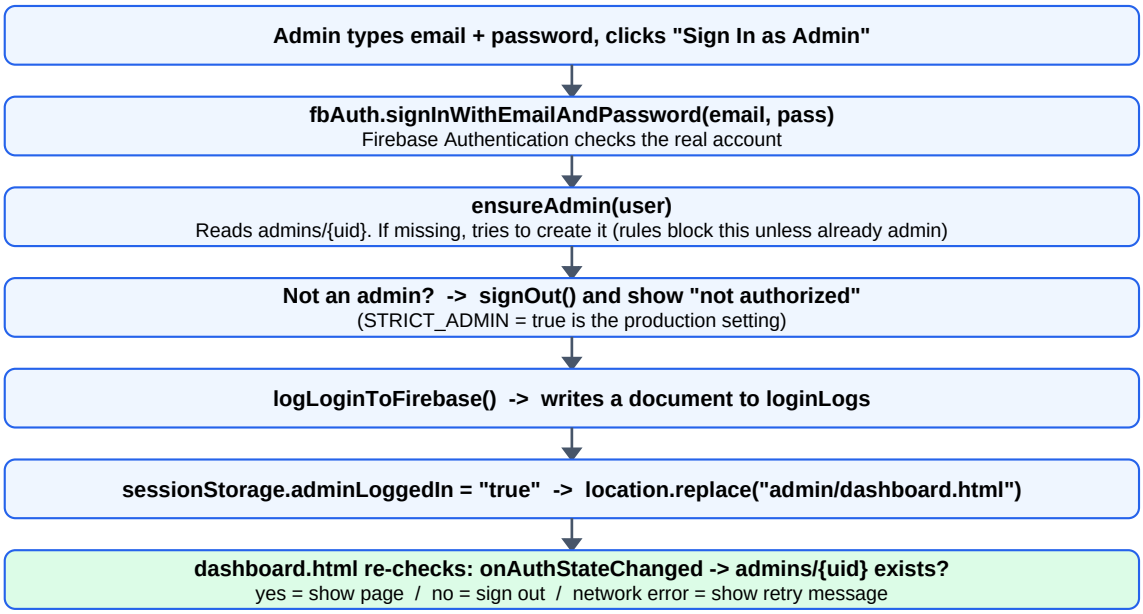
3. System architecture



Everything runs in the browser. There is no custom backend server: the browser talks directly to Firestore and Firebase Auth, and **firestore.rules** is what decides who may read or write each document. The admin dashboard additionally keeps a **copy of every collection in localStorage** so screens render instantly, and keeps that copy fresh with live listeners (section 9).

4. Admin login flow

File: **/index.html** (inline script). Function: **doAdminLogin()**.



- **First admin bootstrap:** create the document *admins/<your-uid>* once in the Firebase console. Rules forbid any client from writing to *admins*.
- On load, the page deletes old keys (adminCreds, adminPassword ...) left in localStorage by an earlier insecure version.
- If the admin is already signed in and returns to the login page, it bounces straight back to the dashboard.
- dashboard.html hides itself (visibility:hidden) until the admin check passes, and has an 8-second watchdog that explains the problem (usually opening the file as file:// instead of through a web server).

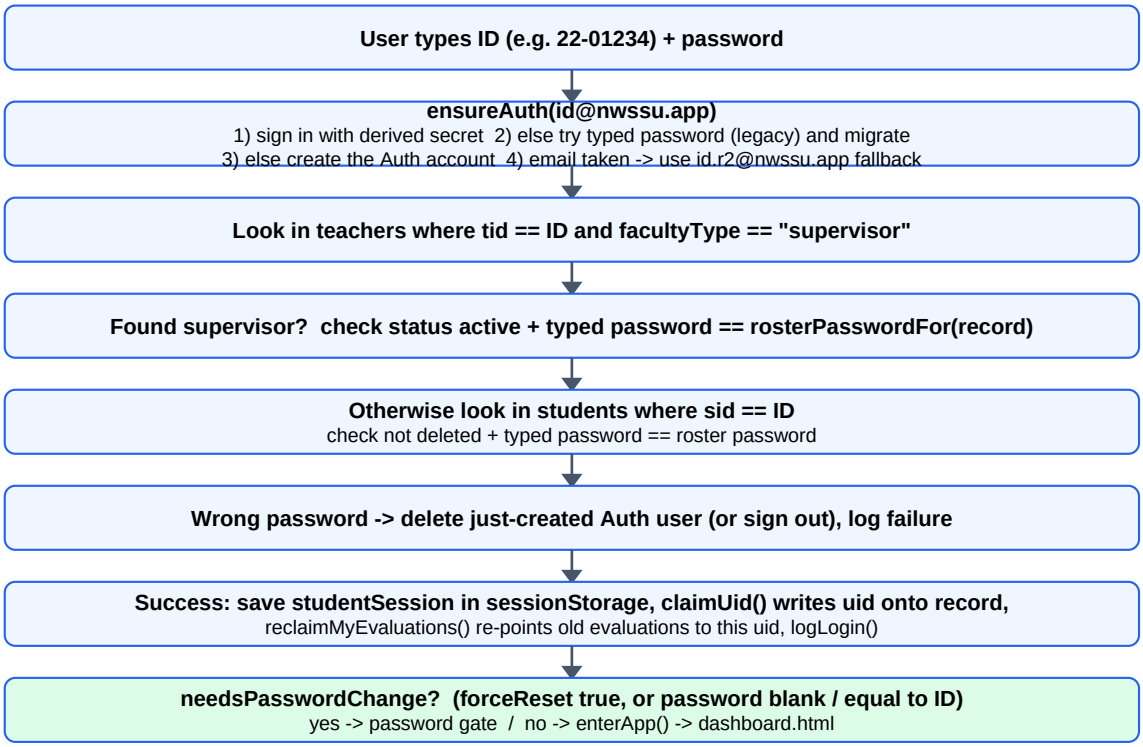
5. Student and supervisor login flow

Files: **Student-user/js/core.js** and **auth.js**. Function: **doLogin()**.

The key idea: there are two passwords.

Password	Where it lives	Who knows it
Roster password (the one the person types)	The *password* field on their students/{doc} or teachers/{doc} record. Blank = their own ID.	The person and the admin.
Firebase Auth secret (hidden)	Derived by authSecretFor(id) = "nwssu:<id>:v1". The Auth email is <id>@nwssu.app.	Nobody types it. The web and Android code compute it.

This lets the admin reset or change a password just by editing the roster field in Firestore, without needing admin access to Firebase Auth.



Default password = the person's ID. New and bulk-imported students get forceReset: true, so on first login they must choose a new password (min 6 characters, not their ID). Inactive students can still log in but see a yellow "inactive" banner.

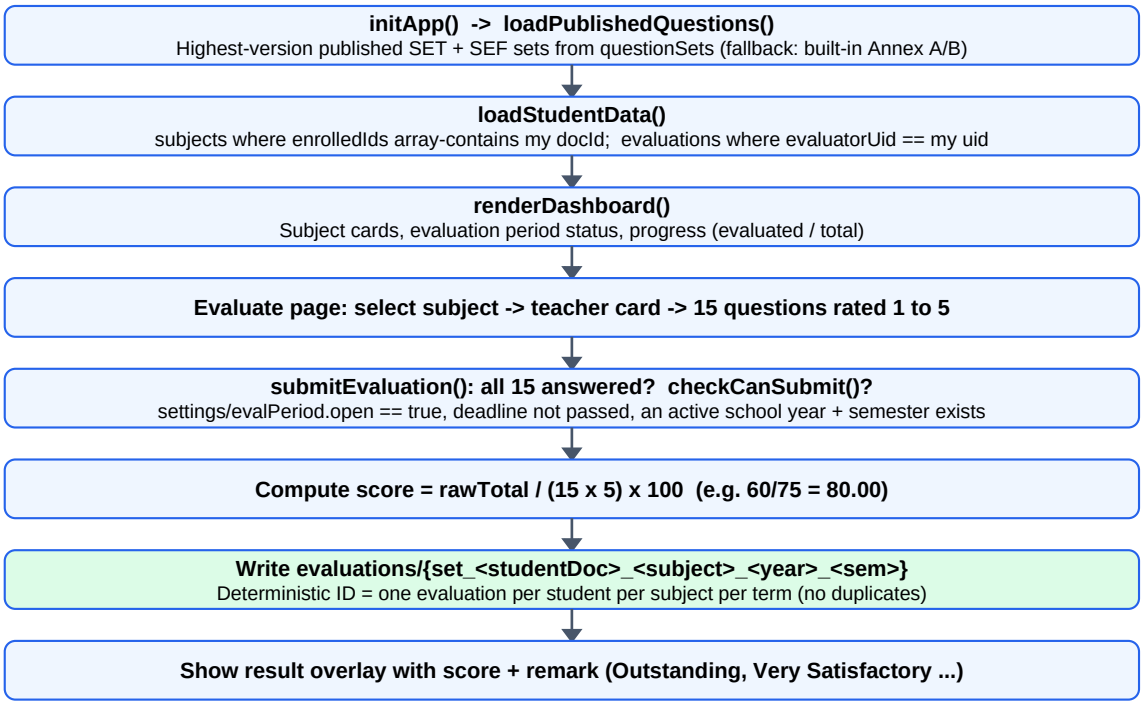
6. Session restore and the password gate

Both portal pages load auth.js. It detects which page it is on (IS_DASHBOARD = the #app element exists) and after *window.load* runs **restoreSession()**:

- **On the login page:** if already signed in with a valid record, go straight to the dashboard.
- **On the dashboard:** no studentSession, or no Firebase user, -> back to login with a clear reason message.
- It re-reads the roster record with _**readRosterDoc()**, which returns *ok / missing / denied / unreachable* (2 retries with timeouts). **Unreachable** (bad wifi) keeps the session and shows a Retry screen instead of logging the person out.
- A hop counter (_hop) stops endless login <-> dashboard redirect loops after 2 hops.
- **submitNewPassword()** (forced gate) and **changePassword()** (Settings) update only *password* and *forceReset* on the person's own record, which is exactly what the security rules allow.

7. Student evaluation flow (SET, Annex A)

File: **Student-user/js/app.js**.



Evaluation record fields

```
{ id, studentId, subjectId, teacherId, evaluatorType: "student", evaluatorUid,
  ratings: { q1: 5, q2: 4, ... q15: 5 }, rawScore, maxRaw: 75, totalScore,
  questionSetId, comment, schoolYear: "2025-2026", semester: "1st Semester",
  timestamp }
```

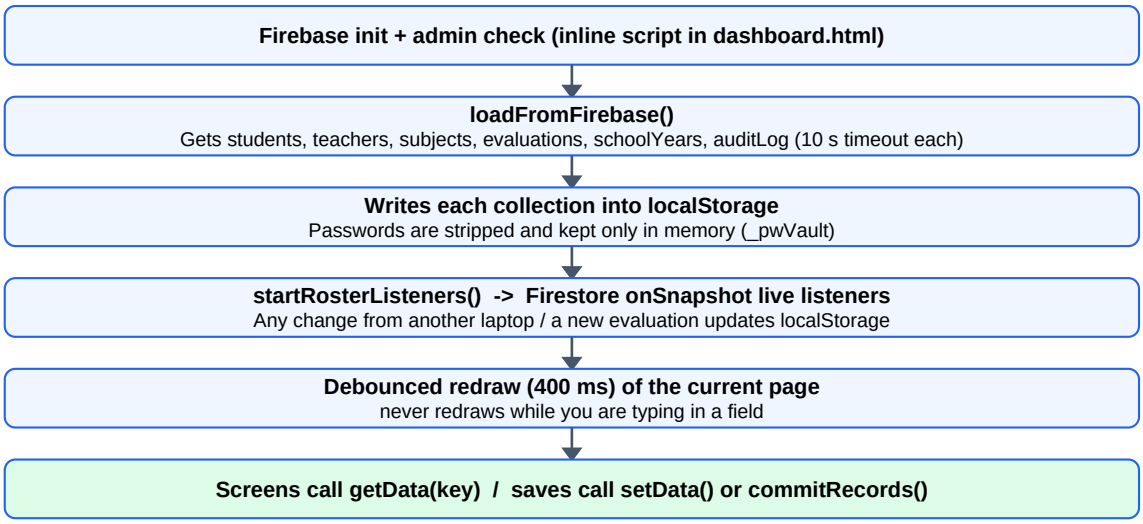
Other student screens: **My Evaluations** (history), **My Feedback** (own ratings + comments; opens a read-only view that shows the question wording of the version that was actually answered), **Profile** and **Settings** (change password). Pages are switched by **showPage(id)**, which toggles the *.active* class on *#page-<id>* divs.

8. Supervisor evaluation flow (SEF, Annex B)

- A supervisor is a **teacher record** with *facultyType: "supervisor"* and *supervisedDepts: [{dept, role}]*.
- initApp() sees userType "supervisor" and calls **renderSupervisorHome()**: **loadSupervisorData()** lists regular faculty in every department they supervise, plus their past SEF submissions.
- **startSef(teacherId)** reuses the Evaluate page with the Annex B questions (items 2, 4 and 9 differ from Annex A).
- **submitSef()** runs the same checks, then writes *evaluations/sef_<supervisorDoc>_<teacher>_<year>_<sem>* with evaluatorType "supervisor" and supervisorId. Submitting again in the same term **updates** the rating instead of duplicating it.

9. Admin dashboard: boot, data layer and sync

The admin dashboard is a single HTML page (**admin/dashboard.html**, ~1,800 lines) with every page as a hidden <div id="page-..."">. Its most important design decision: **screens read from localStorage, not directly from Firestore.**



Data layer functions (admin/js/core/admin-core.js)

Function	What it does
getData(key)	Reads JSON from localStorage. For evaluations, hides ones whose evaluator was deleted (countedEvaluations). For students/teachers, puts passwords back from the in-memory vault.
setData(key, value)	Saves to localStorage and calls syncCollectionToFirestore() to push the collection (used for subjects, schoolYears, settings, questionSets ...). Blocked for students/teachers until passwords have loaded.
commitRecords / writeRecordToCloud	Used for students and teachers. Writes ONLY the changed fields of ONLY the changed record, so two admins on two laptops do not overwrite each other.
commitRecordsBatch	Same, in a Firestore batch (bulk CSV import).
addAudit(action, detail)	Adds an entry to the auditLog (append-only; only new entries are sent).
resetLoginPassword()	Admin reset: sets the roster password (default = ID) and forceReset false.
syncCollectionToFirestore (adminrate.js)	Final version of the sync. Maps keys to collections (departments and customCourses become one doc per department; evalPeriod goes to settings/evalPeriod). Also deletes docs removed locally, but only if the last load was complete (_firestoreLoadClean).

Why the password vault? Earlier versions saved every student's password in localStorage in plain text. Now passwords stay in memory for the session only; closing the tab forgets them. Firestore still holds the real value.

URL routing

admin-router.js wraps showPage() so the current page is kept in the URL hash (e.g. *dashboard.html#reports*). Reload and the Back button therefore return to the same page.

10. Script load order (critical)

The admin scripts are plain global scripts. **When two files define the same function, the one loaded LAST wins.** The order in dashboard.html is therefore part of the program:

#	File	Role
1	shared/js/firebaseAPI.js + inline script	Config, Firebase init, admin check, loadFromFirebase, live listeners
2	core/admin-core.js	getData / setData, password vault, audit, toast, sorting, skeletons

#	File	Role
3	core/admin-scoring.js	All rating maths (weighted SET, SEF, final rating, remarks)
4	core/admin-nav.js	showPage, sidebar, logout, dashboard, school years
5-7	records/admin-students, teachers, subjects	CRUD for people and subjects
8-12	reports/admin-reports, records/admin-bulk, reports/admin-annex, admin-fer, admin-sef	Eval control, analytics, CSV import, Annex C/D, FER, SEF entry, audit page
13	records/admin-courses.js	Courses by department, supervisor migration, page bootstrap
14-18	modules/admindept, adminrate, adminEval, adminfeatures, adminQuestions	Departments; final sync + subject teachers; term logic, exemptions, backup; drill-downs and feedback; question versions
19	core/admin-router.js	MUST be last: wraps the final showPage

Known overrides: *saveSubject, renderSubjects, populateTeacherSelect* in admin-subjects.js are dead code (adminrate.js redefines them). *renderStudents* in admin-students.js is replaced by the one in adminfeatures.js. *syncCollectionToFirestore* in dashboard.html is replaced by the one in adminrate.js. Edit the LAST definition.

11. Admin pages and modules

Page (hash)	Purpose	Main file(s)
#dashboard	Counts and overview	admin-nav.js
#schoolYear	Add school years / semesters, set the ACTIVE semester	admin-nav.js
#students	Add / edit / deactivate / delete students; ID format YY-NNNNN; CSV bulk upload; reset password	admin-students.js, admin-bulk.js, adminfeatures.js
#teachers	Faculty and supervisors (tabs), multi-department supervisor assignments	admin-teachers.js
#subjects	Subjects, curriculum slot (courses + year level), enrolment, one or more teachers per subject (optionally per section)	admin-subjects.js, adminrate.js
#evalControl	Open/close the evaluation period, set the deadline, submission tracking per department / faculty	admin-reports.js
#reports	Faculty and supervisor result tables, filters, term picker, Annex C / D, FER, PDF/Excel export	admin-reports.js, admin-annex.js, admin-fer.js, adminEval.js
#supervisor	Admin-entered SEF (Annex B)	admin-sef.js
#auditLog	Read-only history of admin actions	admin-sef.js
#dept / #deptManage	Department drill-downs and department management (no hard-coded departments)	admindept.js, adminfeatures.js
#feedback	Student comments	adminfeatures.js
Questions modal	Edit and publish new versions of the SET / SEF questions	adminQuestions.js
Backup / Restore	Download / import a JSON backup of all collections	adminEval.js

12. Question sets (versioned instrument)

Evaluations save answers by question ID ({q1: 4, q2: 5 ...}). If a question's wording were edited in place, old answers would silently be reported against the new wording. So **adminQuestions.js** uses versions:

- A **published** set is never edited. Editing creates a **draft**; publishing it creates version n+1 and marks version n **retired**.
- Stored in the **questionSets** collection: { id, instrument: "SET" | "SEF", version, status, questions: [{id, sec, text}], publishedAt, retiredAt, note }.
- The portal and the Android app load the highest published version; every evaluation stores **questionSetId**, so reports show the wording that was actually answered.

- Default questions are CMO 19 Annex A / B **verbatim** (section 4.2 forbids changing the indicators). Sections: A Management of Teaching and Learning (q1-q6), B Content Knowledge, Pedagogy and Technology (q7-q11), C Commitment and Transparency (q12-q15).

13. Scoring and ratings (CMO 19)

All rating maths is in **admin/js/core/admin-scoring.js**, so every report uses the same numbers.

Step 1 - one evaluation

```
score = (sum of 15 ratings) / (15 x 5) x 100          e.g. 63 / 75 x 100 = 84.00
```

Step 2 - weighted SET per faculty: computeWeightedSET() (CMO 8.3)

- Take the faculty's subjects, **excluding** Overload subjects (CMO 4.3) and Lab-school subjects (CMO 8.5).
- For each subject keep only the students that teacher actually teaches (a teacher assigned to sections is rated only by those sections).
- Group the student evaluations by year/section (e.g. "BSIT 4A"). For each section: average score x number of respondents = weighted score.
- Sections or classes with no responses are listed but do not pull the rating down. Exempted students (CMO 8.2) are removed from the roll.
- SET_WEIGHT_BY = "respondents": the count is the students who actually submitted (not the class size).

```
overall SET = sum(avg_section x respondents) / sum(respondents)   (capped at 100)
```

Step 3 - SEF: getSEFForTeacher() (CMO 9.3)

The simple average of all supervisor (SEF) evaluations for that teacher in the term.

Step 4 - final rating: calculateFinalRating() (institutional, not CMO)

```
final = SET x 0.60 + SEF x 0.40      (only when the teacher has BOTH)
otherwise: "Awaiting SEF" or "No evaluations yet"
```

Remarks bands (same in core.js and admin-scoring.js)

Score	Remark	Colour
90 - 100	Outstanding	green
75 - 89.99	Very Satisfactory	blue
60 - 74.99	Satisfactory	amber
50 - 59.99	Fair	orange
below 50	Unsatisfactory	red

- Evaluations from deleted students/supervisors are hidden from every count (isEvaluationHidden). Inactive people still count.
- Printed annexes report SET and SEF separately, as CMO 19 requires; the 60/40 composite is only on internal screens.

14. Term scoping and reports

- **Active term:** the semester marked active under School Year. Every evaluation is stamped with schoolYear + semester.
- **adminEval.js**: _evalMatchesTerm() decides if an evaluation belongs to a term. setReportTerm() lets the admin browse a past term or "ALL". Untagged legacy records appear only in the active term.
- **Evaluation period** (settings/evalPeriod: open, deadline): students can only submit while open. **checkAndAutoFinalize()** runs on a timer; when the deadline is reached it closes the period, writes an audit

entry and runs generateFinalReports() (saved to finalReports).

- **Annex C** (buildAnnexCContent): per-class / per-section weighted SET table; clicking the count shows who has and has not submitted.
- **Annex D** (buildAnnexDContent): summary with SET and SEF, plus the FEDAF development plan (developmentPlans).
- **FER** (generateFERReport, admin-fer.js): institutional Faculty Evaluation Report and trend charts.
- **Signatories** (Prepared / Reviewed / Approved by) are saved once and filled into every printed report; fields remain editable before printing.

15. Firestore data model

Collection	Contents (main fields)	Written by
admins/{uid}	role, email - marks a Firebase user as an administrator	Firebase console only
students	sid (YY-NNNNN), name, course, year, section, dept, password, status, forceReset, deleted, uid	Admin; student may change own password / uid
teachers	tid, name, dept, facultyType ("regular"/"supervisor"), supervisedDepts[{dept, role}], password, status, deleted, uid	Admin; supervisor may change own password / uid
subjects	code, name, courses[], yearLevel, teacherId, teachers[{id, sections[]}], enrolledIds[], loadType, isLabSchool	Admin
evaluations	See section 7. IDs: set_... (student) / sef_... (supervisor)	Students / supervisors (own); admin
questionSets	Versioned SET / SEF instruments	Admin
schoolYears	year, semesters[{label, active}]	Admin
settings	evalPeriod {open, deadline} and other settings docs	Admin
departments / courses	One doc per department code; courses/{DEPT} = {dept, courses[]}	Admin
finalReports, developmentPlans, exemptions	Generated results, FEDAF plans, CMO 8.2 exemptions	Admin
auditLog	Admin actions (append-only)	Admin
loginLogs	username, success, timestamp, userAgent, loginType	Anyone (create only)

Relationships

```
student.id  --in-->  subject.enrolledIds[]
subject.teachers[].id / teacherId  -->  teacher.id
evaluation.studentId -> student.id     evaluation.subjectId -> subject.id
evaluation.teacherId -> teacher.id     evaluation.supervisorId -> teacher.id (SEF)
evaluation.questionSetId -> questionSets.id
evaluation.evaluatorUid  -> Firebase Auth uid (= student.uid / teacher.uid)
```

16. Security rules (firestore.rules)

- **isAdmin()** = signed in AND admins/{uid} exists. Admins can write all management collections.
- Reference data (questionSets, subjects, schoolYears, settings, departments, courses, teachers) is readable by any signed-in user.
- **students**: a student can read only their own record (matched by the ID in their login email). They can update only *password, forceReset, uid*.
- **teachers**: supervisors can likewise update only those three fields on their own record (uses get('tid', '') so missing fields do not break the rule).
- **evaluations**: create only with evaluatorUid == your own uid; read/update your own; delete admin-only. ownsEvaluation() lets a re-created account re-claim old evaluations by changing only evaluatorUid.
- **auditLog** is append-only; **loginLogs** accepts creates from anyone; everything not listed is denied.

17. Important notes and maintenance tips

- **Run through a web server.** Opening the HTML files directly (file://) breaks the login session between pages. Use Firebase Hosting, VS Code Live Server or *python -m http.server*.
- **Deploy:** *firebase deploy* (hosting + rules). Rules changes take effect only after deploying firestore.rules.
- **The Firebase config is repeated in 3 places**: shared/js/firebaseAPI.js, index.html (inline) and Student-user/js/core.js (plus a fallback in admin/dashboard.html). Change all of them if the project changes. The apiKey is a public identifier, not a secret.
- **STRICT_ADMIN** appears in both index.html and admin/dashboard.html; keep them the same.
- **authSecretFor()** in core.js must match the Android app's AuthRepository.kt, or app and web logins will disagree.
- **getRemarks()** exists in both core.js and admin-scoring.js; keep the bands identical.
- **Respect the load order** (section 10) and edit the last definition of an overridden function.
- Firestore may ask for a **composite index** for some queries ("failed-precondition"); the browser console prints a link that creates it.
- Campus networks: both apps enable *experimentalAutoDetectLongPolling* so Firestore works where streaming is blocked.
- Annex wording mirrors the QA office's signed copy and CMO 19; treat changes to annex text as changes to an official document.

Generated from the source code of the Faculty-Eval-System repository.